

WEEK-4

TASK-1 : Process Synchronization and Child Process Monitoring Using waitpid()

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(1);
    }

    printf("Parent is waiting for child PID %d...\n", pid);

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid");
        return 1;
    }
}
```

```

the_boss@DESKTOP-I8C8AJG:~/oss$ ./process_monitor
Parent process started. PID = 1218
Parent is waiting for child PID 1219...
Child process started. PID = 1219
total 196
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 LICENSE
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 Makefile
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 README.md
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:17 a.out
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 assets
drwxr-xr-x 2 the_boss the_boss 4096 Aug 12 14:33 bin
-rw-r--r-- 1 the_boss the_boss 47 Aug 12 14:21 copy.txt
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 docs
-rw-r--r-- 1 the_boss the_boss 1377 Aug 12 13:55 double_quotes.c
-rw-r--r-- 1 the_boss the_boss 599 Aug 12 13:59 escape.c
-rw-r--r-- 1 the_boss the_boss 663 Aug 12 14:01 execute.c
-rw-r--r-- 1 the_boss the_boss 1209 Aug 12 14:19 filecopy.c
-rw-r--r-- 1 the_boss the_boss 443 Aug 11 15:14 fork_example.c
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:22 fork_exec
-rw-r--r-- 1 the_boss the_boss 373 Aug 11 15:22 fork_exec.c
-rw-r--r-- 1 the_boss the_boss 2943 Aug 12 13:37 history.c
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 include
-rw-r--r-- 1 the_boss the_boss 1221 Aug 12 13:24 input.c
-rw-r--r-- 1 the_boss the_boss 1406 Aug 12 13:40 memory.c
-rwxr-xr-x 1 the_boss the_boss 16400 Aug 23 08:02 multiple_children
-rw-r--r-- 1 the_boss the_boss 1047 Aug 23 07:59 multiple_children
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 obj
-rw-r--r-- 1 the_boss the_boss 1741 Aug 12 13:47 parse_tree.c
-rw-r--r-- 1 the_boss the_boss 736 Aug 12 13:45 parser.c
-rw-r--r-- 1 the_boss the_boss 976 Aug 12 14:33 process.c
-rwxr-xr-x 1 the_boss the_boss 16312 Aug 23 08:23 process_monitor
-rw-r--r-- 1 the_boss the_boss 935 Aug 23 08:22 process_monitor.c
-rw-r--r-- 1 the_boss the_boss 23 Aug 12 14:25 sample.txt
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 scripts
-rw-r--r-- 1 the_boss the_boss 475 Aug 12 13:18 shell.c
-rw-r--r-- 1 the_boss the_boss 802 Aug 12 13:53 single_quotes.c
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 src
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 tests
-rw-r--r-- 1 the_boss the_boss 1047 Aug 23 07:44 wait_waitpidn
-rwxr-xr-x 1 the_boss the_boss 16256 Aug 23 08:17 zombie

```

REPORT: I created a parent and child process using `fork()` and used `waitpid()` to synchronize the child process with the parent. From the output, I observed that the parent waited until the child process completed and then checked its exit status. I understood how `waitpid()` helps the parent monitor and synchronize with a particular child process.

Task-2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_PATH 4096

int main(int argc, char *argv[])
{
    char *path;
    char *path_copy;
    char *directory;
    char full_path[MAX_PATH];

    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("strdup");
        return 1;
    }

    directory = strtok(path_copy, ":");
```

OUTPUT:

```
the_boss@DESKTOP-I8C8AJG:~/oosp$ nano path_resolver.c
the_boss@DESKTOP-I8C8AJG:~/oosp$ gcc path_resolver.c -o path_resolver
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./path_resolver
Usage: ./path_resolver <command>
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./path_resolver ls
Command found: /usr/bin/ls
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./path_resolver gcc
Command found: /usr/bin/gcc
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./path_resolver hello123
Command not found: hello123
```

Report: I implemented a program to search for commands using the PATH environment variable. The program searches the directories in PATH and checks whether the command exists and has executable permission. I tested the program using the ls command and it successfully located the executable. I also tested a missing command to check the error handling. I understood how the shell searches for executable commands using the PATH variable.

Task-3:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;
            char *value;

            i++;

            while (isalnum((unsigned char)input[i]) ||
                    input[i] == '_')
            {
                if (k < 99)
                    variable[k++] = input[i];

                i++;
            }

            variable[k] = '\0';

            value = getenv(variable);

            if (value != NULL)
            {
```

OUTPUT:


```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano variable_expand.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc variable_expand.c -o variable_expand
the_boss@DESKTOP-I8C8AJG:~/oss$ ./variable_expand
Enter text: My home directory is $HOME
Expanded text: My home directory is /home/the_boss
the_boss@DESKTOP-I8C8AJG:~/oss$ Current user is $USER
Current: command not found
```

Report: I implemented variable expansion in the shell by detecting variable references using \$ and retrieving their values from the environment. I tested defined and undefined variables and observed that defined variables are replaced with their actual values, while undefined variables are reported. I understood how environment variables can be expanded and updated in the input string. To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logical

Task-4:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic



the_boss@DESKTOP-I8C8AJG: X



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef void (*builtin_func)(char *);

void builtin_pwd(char *args)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
}

void builtin_cd(char *args)
{
    if (args == NULL || strlen(args) == 0)
    {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args) != 0)
        perror("cd");
}

void builtin_exit(char *args)
{
    printf("Exiting shell...\n");
    exit(0);
}

typedef struct
{
    char *name;
    builtin_func function;
}
```

```
the_boss@DESKTOP-I8C8AJG:~/ossp$ nano builtin_dispatch.c
the_boss@DESKTOP-I8C8AJG:~/ossp$ gcc builtin_dispatch.c -o builtin_dispatch
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./builtin_dispatch
-bash: ./builtin_dispatch: No such file or directory
the_boss@DESKTOP-I8C8AJG:~/ossp$ pwd
/home/the_boss/ossp
```

Report: I implemented a dispatch system for built-in shell commands such as cd, pwd, echo, help, and exit. I tested the commands and also checked an invalid command to verify error handling. I understood that built-in commands are executed within the shell process and that commands like cd can maintain the shell's current state.